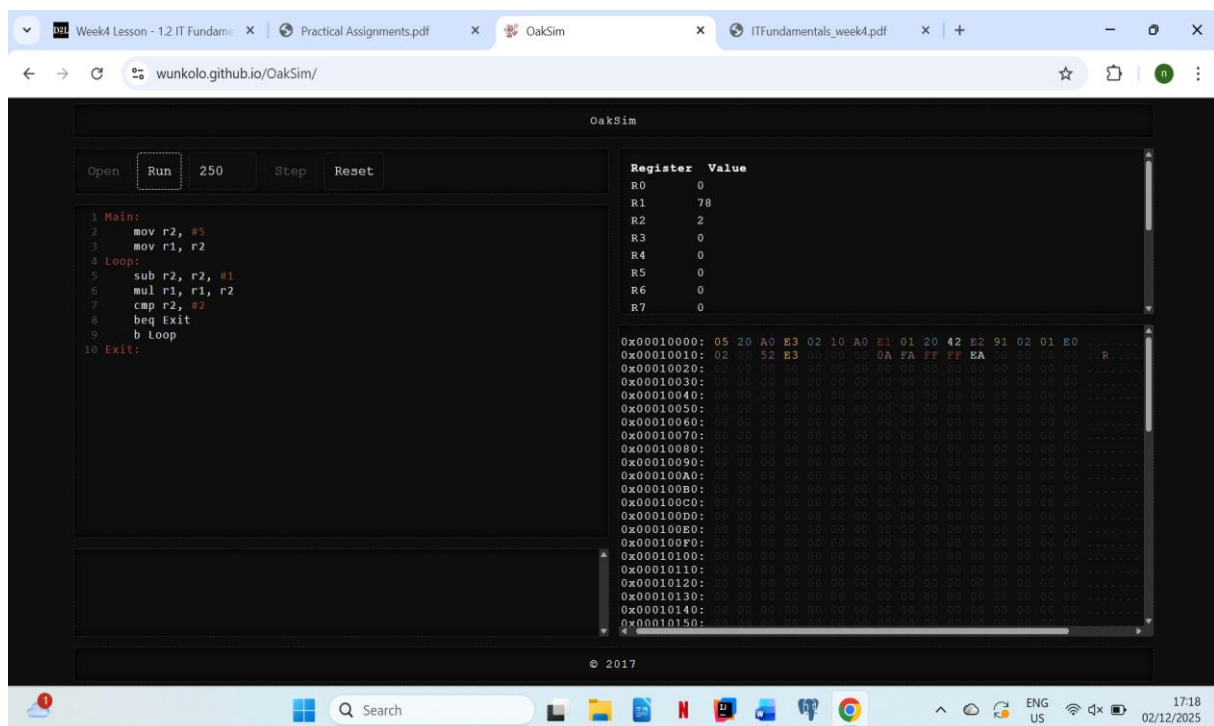


Template Week 4 – Software

Student number:556707

Assignment 4.1: ARM assembly

Screenshot of working assembly code of factorial calculation:



Assignment 4.2: Programming languages

Take screenshots that the following commands work:

javac --version

java --version

gcc --version

python3 --version

bash --version

```
nassima@biede: ~  
update-alternatives: using /usr/lib/jvm/java-21-openjdk-amd64/bin/jconsole to pr  
vide /usr/bin/jconsole (jconsole) in auto mode  
nassima@biede:~$ javac --version  
javac 21.0.9  
nassima@biede:~$ java --version  
openjdk 21.0.9 2025-10-21  
OpenJDK Runtime Environment (build 21.0.9+10-Ubuntu-124.04)  
OpenJDK 64-Bit Server VM (build 21.0.9+10-Ubuntu-124.04, mixed mode, sharing)  
nassima@biede:~$ python3 --version  
Python 3.12.3  
nassima@biede:~$ bash --version  
GNU bash, version 5.2.21(1)-release (x86_64-pc-linux-gnu)  
Copyright (C) 2022 Free Software Foundation, Inc.  
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>  
  
This is free software; you are free to change and redistribute it.  
There is NO WARRANTY, to the extent permitted by law.  
nassima@biede:~$ gcc --version  
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0  
Copyright (C) 2023 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  
  
nassima@biede:~$
```

Assignment 4.3: Compile

Which of the above files need to be compiled before you can run them?

- ❑ **Fibonacci.java (Java source code) — needs to be compiled to Java bytecode.**
- ❑ **fib.c (C source code) — needs to be compiled to machine code.**
- ❑ **fib.py (Python source code) — does not need to be compiled manually; it's interpreted.**
- ❑ **fib.sh (Bash script) — does not need to be compiled; it's interpreted by the shell.**

Which source code files are compiled into machine code and then directly executable by a processor?

fib.c (C source code) — compiled to machine code (e.g., .out or executable binary)

Which source code files are compiled to byte code?

Fibonacci.java — compiled to Java bytecode (.class files), which runs on the Java Virtual Machine (JVM).

Which source code files are interpreted by an interpreter?

fib.py — interpreted by the Python interpreter.

fib.sh — interpreted by the Bash shell.

These source code files will perform the same calculation after compilation/interpretation. Which one is expected to do the calculation the fastest?

- **fib.c (compiled C program) — generally the fastest because it's compiled directly into machine code and runs natively.**
- **Java bytecode runs on JVM which adds some overhead.**
- **Python and Bash are interpreted and generally slower than compiled languages.**

How do I run a Java program?

compile the Java file:

javac Fibonacci.java

This creates Fibonacci.class.

Run the compiled program:

java Fibonacci

How do I run a Python program?

python3 fib.py

How do I run a C program?

Compile the C source:

gcc fib.c -o fib

This creates an executable file named fib.

Run the executable:

./fib

How do I run a Bash script?

bash fib.sh

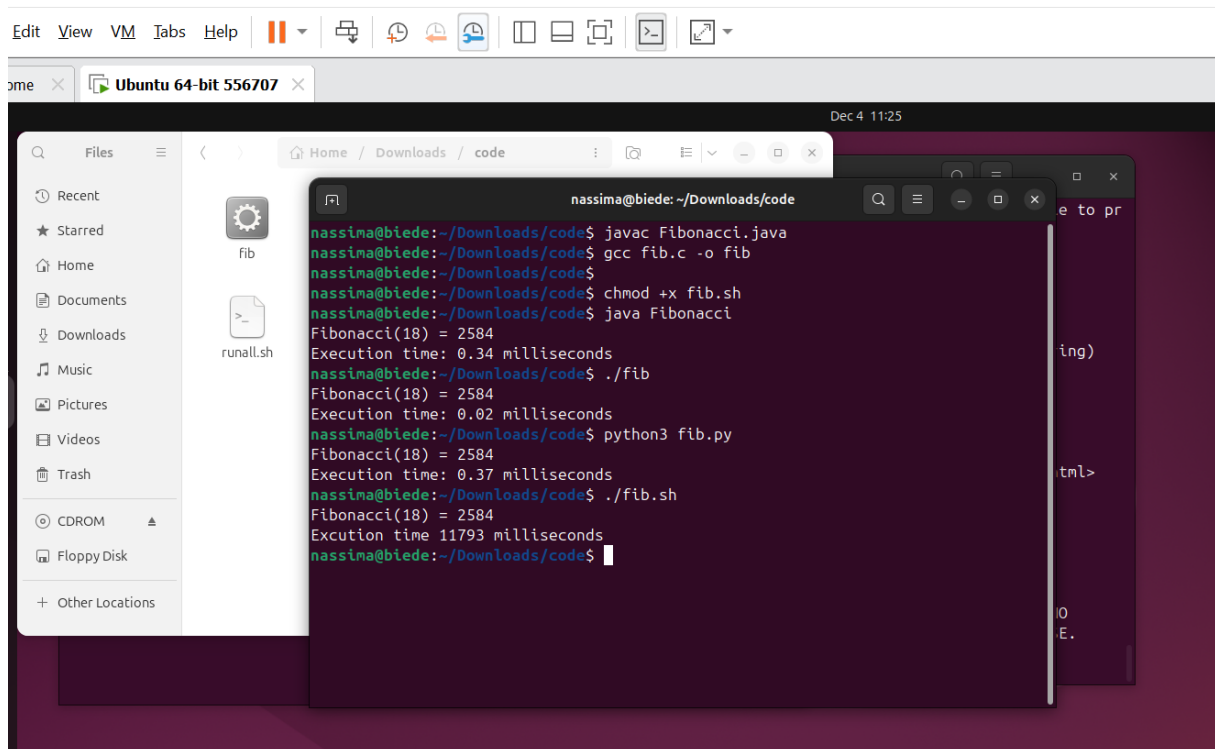
If I compile the above source code, will a new file be created? If so, which file?

- **Java: Compiling Fibonacci.java creates Fibonacci.class.**
- **C: Compiling fib.c creates an executable file (default a.out or as you specify, e.g., fib).**
- **Python: No manual compilation needed; no new file is created unless Python bytecode files (.pyc) are auto-generated.**
- **Bash: No compilation, no new files created.**

Take relevant screenshots of the following commands:

- Compile the source files where necessary
- Make them executable
- Run them
- Which (compiled) source code file performs the calculation the fastest?

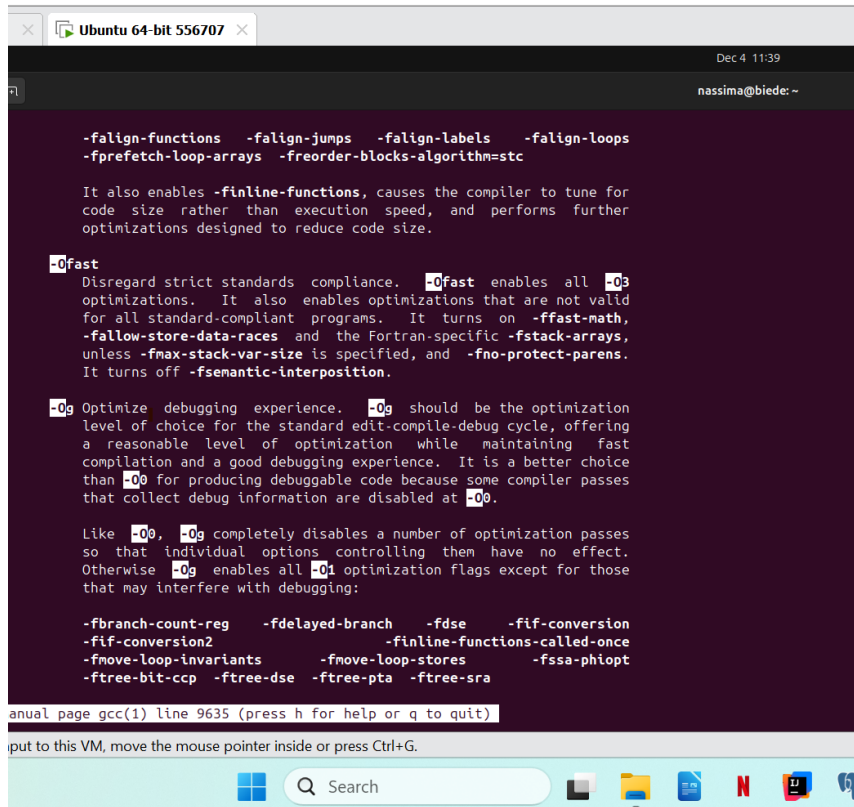
C program is the fastest by a significant margin.



Assignment 4.4: Optimize

Take relevant screenshots of the following commands:

- a) Figure out which parameters you need to pass to **the gcc** compiler so that the compiler performs a number of optimizations that will ensure that the compiled source code will run faster. **Tip!** The parameters are usually a letter followed by a number. Also read **page 191** of your book, but find a better optimization in the man pages. Please note that Linux is case sensitive.



The screenshot shows a terminal window titled 'Ubuntu 64-bit 556707' with the user 'nassima@biede: ~'. It displays the man page for gcc(1), specifically the section for optimization options. The visible text includes:

```
-falign-functions -falign-jumps -falign-labels -falign-loops
-fprefetch-loop-arrays -fpreorder-blocks-algorithm=stc

It also enables -finline-functions, causes the compiler to tune for
code size rather than execution speed, and performs further
optimizations designed to reduce code size.

-Ofast
Disregard strict standards compliance. -Ofast enables all -O3
optimizations. It also enables optimizations that are not valid
for all standard-compliant programs. It turns on -ffast-math,
-fallow-store-data-races and the Fortran-specific -fstack-arrays,
unless -fmax-stack-var-size is specified, and -fno-protect-parens.
It turns off -fsemantic-interposition.

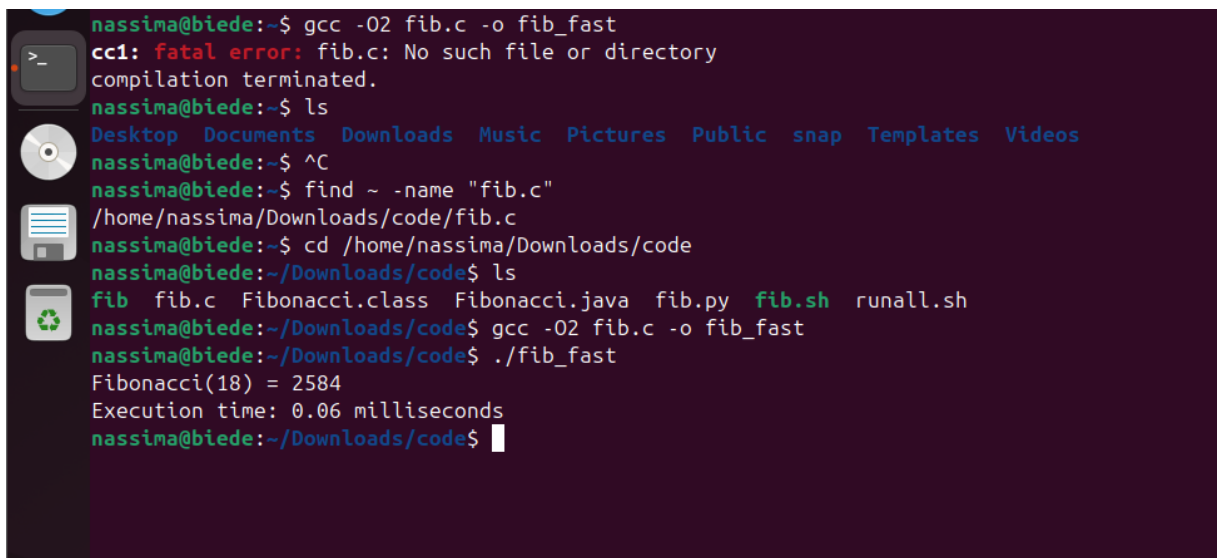
-Og
Optimize debugging experience. -Og should be the optimization
level of choice for the standard edit-compile-debug cycle, offering
a reasonable level of optimization while maintaining fast
compilation and a good debugging experience. It is a better choice
than -O0 for producing debuggable code because some compiler passes
that collect debug information are disabled at -O0.

Like -O0, -Og completely disables a number of optimization passes
so that individual options controlling them have no effect.
Otherwise -Og enables all -O1 optimization flags except for those
that may interfere with debugging:

-fbranch-count-reg -fdelayed-branch -fdse -fif-conversion
-fif-conversion2 -finline-functions-called-once
-fmove-loop-invariants -fmove-loop-stores -fssa-phiopt
-ftree-bit-ccp -ftree-dse -ftree-pta -ftree-sra
```

annual page gcc(1) line 9635 (press h for help or q to quit)

- b) Compile **fib.c** again with the optimization parameters



The screenshot shows a terminal window with the following commands and output:

```
nassima@biede:~$ gcc -O2 fib.c -o fib_fast
cc1: fatal error: fib.c: No such file or directory
compilation terminated.
nassima@biede:~$ ls
Desktop Documents Downloads Music Pictures Public snap Templates Videos
nassima@biede:~$ ^C
nassima@biede:~$ find ~ -name "fib.c"
/home/nassima/Downloads/code/fib.c
nassima@biede:~$ cd /home/nassima/Downloads/code
nassima@biede:~/Downloads/code$ ls
fib fib.c Fibonacci.class Fibonacci.java fib.py fib.sh runall.sh
nassima@biede:~/Downloads/code$ gcc -O2 fib.c -o fib_fast
nassima@biede:~/Downloads/code$ ./fib_fast
Fibonacci(18) = 2584
Execution time: 0.06 milliseconds
nassima@biede:~/Downloads/code$
```

- c) Run the newly compiled program. Is it true that it now performs the calculation faster?

```

/home/nassima/Downloads/code/fib.c
nassima@biede:~$ cd /home/nassima/Downloads/code
nassima@biede:~/Downloads/code$ ls
fib  fib.c  Fibonacci.class  Fibonacci.java  fib.py  fib.sh  runall.sh
nassima@biede:~/Downloads/code$ gcc -O2 fib.c -o fib_fast
nassima@biede:~/Downloads/code$ ./fib_fast
Fibonacci(18) = 2584
Execution time: 0.06 milliseconds
nassima@biede:~/Downloads/code$ gcc -O0 fib.c -o fib_slow
nassima@biede:~/Downloads/code$ gcc -O2 fib.c -o fib_fast
nassima@biede:~/Downloads/code$ /usr/bin/time -p ./fib_slow
/usr/bin/time -p ./fib_fast
Fibonacci(18) = 2584
Execution time: 0.03 milliseconds
real 0.00
user 0.00
sys 0.00
Fibonacci(18) = 2584
Execution time: 0.01 milliseconds
real 0.00
user 0.00
sys 0.00
nassima@biede:~/Downloads/code$

```

- d) Edit the file **runall.sh**, so you can perform all four calculations in a row using this Bash script. So the (compiled/interpreted) C, Java, Python and Bash versions of Fibonacci one after the other.

Assignment 4.5: More ARM Assembly

Like the factorial example, you can also implement the calculation of a power of 2 in assembly. For example you want to calculate $2^4 = 16$. Use iteration to calculate the result. Store the result in r0.

Main:

```
mov r1, #2
```

```
mov r2, #4
```

Loop:

End:

Complete the code. See the PowerPoint slides of week 4.

Screenshot of the completed code here.

Ready? Save this file and export it as a pdf file with the name: **week4.pdf**

